



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

Проект

ЗА ОСВОБОЖДАВАНЕ ОТ ИЗПИТ

Дисциплина: „Програмни среди”

Тема: Малка система за rent-a-car, управление на коли, клиенти, локации,
наеми на коли

Изготвил:

Василена Емилова Милчова

Фак. № 121223030

Група: 38

III курс, КСИ

e-mail: vamilchova@tu-sofia.bg

София, 2026

Съдържание

Съдържание.....	2
Въведение.....	3
Функционалности на проекта.....	3
View технологии и СУБД.....	3
WPF срещу Blazor.....	3
SQLite срещу SQL Server.....	4
Средата и библиотеките.....	5
Описание на реализацията.....	6
Архитектура.....	6
Структура.....	6
Описание по слоеве.....	8
Ключови типове.....	9
Преизползване на ViewModel-ите.....	10
Преизползването на СУБД.....	10
Ключови откъси от кода.....	11
Пример за използване на функционалностите.....	14
Добавяне на нов запис.....	14
Премахване на запис.....	15
Редактиране на запис.....	16
Филтриране на записите.....	16
Смяна на базата данни.....	16
Заклучение.....	17
Източници.....	17

Въведение

Проектът представлява малка информационна система за rent-a-car фирма, която позволява да се управляват: автомобили, клиенти, локации и наемане на коли. Програмата е разработена с цел да се покажат добри практики за разделяне на слоеве (Model, ViewModel, View), преизползване на логика и работа с повече от една UI технология и повече от една СУБД с един и същи код.

Функционалности на проекта

От функционална гледна точка приложението позволява да се разглежда списък с автомобили, да се филтрират по различни критерии (марка, модел, категория, статус, цена за наемане/ден, локация), да се създават нови записи, да се редактират съществуващи и да се изтриват. За колите се пази информация за регистрационен номер, марка, модел, категория, дневен наем, статус и връзка към локация, на която автомобилът се намира. Локациите се seed-ват в базата при създаване на схемата, за да има реалистични данни още от първото стартиране. За клиентите се пази тяхното име, имейл и номер на шофьорска книжка. Таблицата за създаване на нов наем(*rental*) за кола свързва колите и клиенти, като позволява да се пазят също стартова и крайна дата на наемането, както и цена за целия *rental*, която се изчислява автоматично.

Специфичното в задачата е, че има два различни потребителски интерфейса върху една и съща логика и база данни: *WPF* - десктоп приложение и *Blazor Server* - уеб приложение. И двете използват едни и същи *ViewModel*-и от проект *RentACar.ViewModel*, както и един и същи модел и репозитори от *RentACar.Model*. Това дава възможност да се демонстрира, че *ViewModel*-ът е независим от конкретната *UI* технология – *WPF* достъпва методите му през *code-behind*[\[1\]](#) и *data binding*, а *Blazor* ги вика директно от *Razor* компонентите чрез *dependency injection*.

За данните са използвани две различни СУБД чрез *Entity Framework Core* [\[3\]](#): *SQLite* и *SQL Server LocalDB*. В слоя *Model* има *DbContextFactory*, която избира кой *provider* да се използва чрез свойството *Provider* и съответно конфигурира *UseSqlServer* или *UseSqlite* при създаване на контекста.

View технологии и СУБД

WPF срещу Blazor

При работата по проекта *WPF* се усеща повече като „класическа“ десктоп технология, в която интерфейсът се описва в *XAML*, а логиката стои или в *code-behind* [\[1\]](#), или във *ViewModel*. Най-много помогна това, че може директно да се подава *DataContext* и да се използва двупосочен *binding* – например когато се свърже

CarPageView към *CarPageViewModel*, бутоните просто викат методи от *ViewModel*-а и списъкът от коли се обновява автоматично, защото е *ObservableCollection* [2]. При *WPF* се усеща по-силно разделението *View/ViewModel*, защото *XAML* и *C#* са в отделни файлове и има ясно място за визуалната част и място за логиката.

Въпреки това, по лично мнение *WPF* се оказа не до там по-трудната, но по-досадна и “неблагодарна” технология от двете: *XAML* е тежък за четене и писане - дребна синтактична грешка често чупи целия UI, а правенето на „красив“ дизайн изисква много време, шаблони и стилове, което не изглежда да си заслужава усилието.

Blazor Server е по-лесен за експериментиране и е уеб базиран. *Razor* страниците комбинират *HTML+CSS* и *C#* в един файл, а през *dependency injection* може директно да се инжектира *CarPageViewModel* и да се извикват методите му вътре в компонента. *HTML* се чете по-лесно от *XAML* и за прости таблици и форми е по-бърз за написване – реално страницата *Cars.razor* се получи с много по-малко усилие от *WPF* екрана, като и двата поддържат еднакви функционалности, но и разположението на елементите е много по-просто в *Blazor* спрямо *WPF*, което помогна да се създаде по-бързо, по-удобен за използване и по-красив UI.

Това, което помогна да се разбере с използване на две различни технологии за потребителския интерфейс е практическото упражнение, например да се направи същата страница в *Blazor* с функциите от *ViewModel*, които преди това бяха използвани за *WPF* страницата. Например при зарежда колите с *await CarPage.LoadAsync()* и при натискане на бутон „Save“ извиква същото *Edit.SaveAsync()*, което се ползва и от *WPF*.

Разликата във усещането е, че при *WPF* постоянно трябва да се мисли *XAML*, ресурси, *layout-панели* и стилове, докато при *Blazor* повече се мисли за компоненти и *C#* логиката. За този конкретен проект, в който акцентът е върху архитектурата, а не върху перфектния дизайн, и *Blazor* и *WPF* практически вършат работа, както и това, че са далеч един от друг по начин на работа и имплементация е идеята защо точно тях използвах за проекта. Най-полезната информация тук беше как да регистрират *ViewModel*-ите и *DbContext* в *DI (Dependency Injection)* и после да ги *@inject*-на в *Razor* компонента, вместо да се създават нови такива. Това реално беше ключът *Blazor* да работи по същия начин като *WPF*, но в уеб среда.

SQLite срещу SQL Server

SQLite се оказа много удобен за бърз старт: достатъчен е *connection string* от вида *Data Source=...* и *EF Core* [3] създава файла при първия *EnsureCreated()*. Реалният проблем не беше в самия *SQLite*, а в това, че *WPF* и *Blazor* автоматично създават *.db* файла в своите *bin* папки и така се получиха две отделни бази. Това стана очевидно, когато в *WPF* имаше записи, а в *Blazor* беше празно. Решението беше да се напише *DbPathHelper.cs*, който да изчислява общ път към *SharedData\rentacar.db* в пътя над проектите, така че и двете приложения да работят с един и същи файл.

SQL Server LocalDB, нямаше такъв проблем, защото не създава файловете в проекта, а работи като инстанция на сървър със собствено име *((localdb)\MSSQLLocalDB)* и изисква да има инсталирана среда. При него трудността беше, че ако не се извика *UseSqlServer(cs)* в *DbContextOptionsBuilder*, се получава грешка „*No database provider has been configured*“, дори да има правилен *connection string* в променлива. Това помогна да стане ясно, че при *EF Core* [3] не е достатъчно да има текстов *connection string* – трябва и правилен *provider*, и той се включва чрез *UseSqlServer* или *UseSqlite* в един централен метод (*DbContextFactory.Create* или *AddDbContext*). Другата разлика, усетена на практика, е че при *SQL Server* трябва да има *LocalDB* инстанцията, която да е налична и стартирана, докато при *SQLite* това не е фактор.

Средата и библиотеките

Голяма част от усилието беше именно в настройка на средата и библиотеките, а не в самото писане на кода. Проектът е изграден върху *.NET 8*, което позволява едновременно използване на модерната *Blazor Server* платформа и актуалните версии на *Entity Framework Core* [3] за работа с бази данни. Това означава, че всички библиотеки и шаблони са съобразени с *.NET 8 SDK*.

За да работи слой за данни (*Model*) с двете избрани СУБД – *SQLite* и *SQL Server LocalDB* – беше необходимо да се добавят няколко основни *NuGet* пакета за *Entity Framework Core*:

- ***Microsoft.EntityFrameworkCore*** – базовият пакет за *EF Core*, необходим за дефиниране на *DbContext*, *DbSet<T>*, *Fluent API* конфигурации и асинхронни операции като *ToListAsync*, *SaveChangesAsync* и др.
- ***Microsoft.EntityFrameworkCore.Sqlite*** – *provider* за *SQLite*, който позволява да се извика *options.UseSqlite("Data Source=...")* в *DbContextFactory* и в *Blazor Program.cs*, така че и *WPF*, и *Blazor* да работят с един и същи *.db* файл в *SharedData*.
- ***Microsoft.EntityFrameworkCore.SqlServer*** – *provider* за *SQL Server*, използван за работа с *LocalDB*, при избор на *Provider = "SqlServer"* във фабриката.

От страна на *Blazor* и *ASP.NET Core* са използвани стандартните пакети, които идват с *Blazor Server* проекта под *.NET 8*, например:

- *Microsoft.AspNetCore.Components.Web*
- *Microsoft.AspNetCore.Components.WebAssembly.Server* за хостинг
- *DI* контейнерът, който се конфигурира чрез:
 - *builder.Services.AddDbContext*
 - *AddScoped<IUnitOfWork*
 - *UnitOfWork>()*

- `AddScoped<CarPageViewModel>()`.

Тази конфигурация подsigурява, че всеки *HTTP request* и всяка *Blazor* сесия получава собствен екземпляр на *DbContext* и *ViewModel*, създадени по правилния начин според избрания *provider*.

По този начин комбинацията от *.NET 8*, *EF Core provider-ume* за *SQLite* и *SQL Server* и *DI* инфраструктурата [5] на *ASP.NET Core* позволяват един и същ домейн модел и *ViewModel* слой да работят едновременно в *WPF* десктоп приложение и *Blazor Server* уеб приложение, върху споделена база данни.

Описание на реализацията

Архитектура

Архитектурата е разделена на три основни слоя: *Model*, *ViewModel* и *View*, като *Model* и *ViewModel* са общи за *WPF* и *Blazor*, а *View* слой е различен за двете технологии (*WPF* и *Blazor*).

- Слой *Model* е отговорен за данните и достъпа до базата
- Слой *ViewModel* съдържа логиката зад *UI*-а: държи състояние за страниците, изпълнява операции през *IUnitOfWork* и подготвя данните за визуализация, без да зависи от *WPF* или *Blazor*.
- Слой *View* реализира конкретния потребителски интерфейс:
 - *WPF* – *XAML* форми и *code-behind*, които правят *data binding* към *ViewModel*-ите.
 - *Blazor* – *Razor* компоненти, които инжектират същите *ViewModel*-и през *DI* [4] и ги използват за визуализация и обработка на събития.

Това разделение позволява да се сменя *UI* технологията, без да се пренаписва бизнес логиката и достъпа до данни.

Структура

```
RentACar/                                <- solution root
|
├─ SharedData/                            <- общата папка за SQLite файла
│   └─ rentacar.db
|
├─ RentACar.Model/                        <- Model слой (данни и EF Core)
│   └─ Entities/
│       ├── Car.cs
│       ├── Customer.cs
│       ├── Location.cs
│       └─ Rental.cs
|
└─ Data/
```

- └─ RentACarDbContext.cs
 - └─ DbContextFactory.cs
 - └─ DbPathHelper.cs
- └─ Interfaces/
 - └─ IUnitOfWork.cs
 - └─ IRepository<T>.cs (интерфейси)
- └─ Repositories/
 - └─ CarsRepository.cs
 - └─ CustomersRepository.cs
 - └─ LocationsRepository.cs
 - └─ RentalsRepository.cs
- └─ UnitOfWork/
 - └─ UnitOfWork.cs
- └─ [RentACar.ViewModel/](#) <- ViewModel слой
 - └─ ViewModels/
 - └─ CarPageViewModel.cs
 - └─ CarSearchViewModel.cs
 - └─ CarListViewModel.cs
 - └─ CarEditViewModel.cs
 - └─ CustomerPageViewModel.cs
 - └─ ...
 - └─ MainViewModel.cs
- └─ [RentACar.Wpf/](#) <- WPF десктоп приложение
 - └─ App.xaml
 - └─ App.xaml.cs
 - └─ MainWindow.xaml
 - └─ MainWindow.xaml.cs
 - └─ Views/
 - └─ CarPageView.xaml
 - └─ CarPageView.xaml.cs
 (другие страницы по същия модел)
- └─ [RentACar.Blazor/](#) <- Blazor Server веб приложение
 - └─ Program.cs
 - └─ App.razor
 - └─ Components/
 - └─ App.razor (auto-generated hosting component)
 - └─ MainLayout.razor
 - └─ MainLayout.razor.css
 - └─ NavMenu.razor
 - └─ NavMenu.razor.css
 - └─ Pages/
 - └─ Customers.razor
 - └─ Cars.razor <- използва CarPageViewModel
 - └─ ...
 - └─ DatabaseProviderService.cs

Описание по слоеве

Model (RentACar.Model):

➤ **Entities:**

- *Car, Customer, Location, Rental* – домейн класове, които описват основните обекти от системата и навигационните им свойства.

➤ **Data:**

- *RentACarDbContext* – EF Core контекст с *DbSet<Car>*, *DbSet<Customer>*, *DbSet<Location>*, *DbSet<Rental>*, конфигурация на релации и *seed*-ване на локации.
- *DbContextFactory* – фабрика със свойство *Provider* и метод *Create()*, който конфигурира контекста за *SQL Server* или *SQLite*.
- *DbPathHelper* – помощен клас, който намира общия път към *SharedData\rentacar.db*.

➤ **Interfaces:**

- *IUnitOfWork*, интерфейси за репозитори (например *ICarRepository*).

➤ **Repositories:**

- *CarsRepository, CustomersRepository, LocationsRepository, RentalsRepository* – инкапсулират заявки към EF Core (включително филтри и *Include*-ове).

➤ **UnitOfWork:**

- *UnitOfWork* – държи един *RentACarDbContext* и предоставя свойства за достъп до репозитори и метод *SaveChangesAsync()*.

ViewModel (RentACar.ViewModel):

- *CarPageViewModel* – главният *ViewModel* за страницата с коли; съдържа *Search, List, Edit* и метод *LoadAsync()*.
- *CarSearchViewModel* – държи филтрите (*Category, Status, DailyRateMin/Max, MakeContains, ModelContains, LocationId*) и има метод *SearchAsync()*, който вика *Cars.SearchAsync(...)* през *IUnitOfWork* и пълни списъка с коли.
- *CarListViewModel* – съдържа *ObservableCollection<Car>* и свойството *SelectedCar*, плюс методи *SetCars* и *Clear*.
- *CarEditViewModel* – управлява редакцията: *Current* (текуща кола), *Locations* (dropdown), *SelectedLocation, HasCurrent*, методи *LoadLocationsAsync, StartNew, StartEdit, SaveAsync, DeleteAsync*.

- Други странични *ViewModel*-и (например за клиенти) следват същия модел – страница, търсене, изкарване на данните, редакция.

View – WPF (*RentACar.Wpf*):

- *MainWindow* – създава контекста чрез *DbContextFactory.Create()*, инициализира *UnitOfWork* и *MainViewModel*, задава *DataContext* и в *Loaded* извиква началното зареждане на данни.
- *CarPageView.xaml/.cs* – визуализира таблицата с коли и формата за редакция; в *code-behind* бутоните *Search*, *New*, *Edit*, *Save*, *Delete* логически просто викат методите от *CarPageViewModel*.

View – Blazor (*RentACar.Blazor*):

- *Program.cs* – настройва DI: *AddDbContext<RentACarDbContext>(...)*, *AddScoped<IUnitOfWork, UnitOfWork>()*, *AddScoped<CarPageViewModel>()*, и извиква *Database.EnsureCreated()* при стартиране.
- *Cars.razor* – инжектира *CarPageViewModel*, в *OnInitializedAsync* вика *CarPage.LoadAsync()* и *CarPage.Edit.LoadLocationsAsync()*, визуализира таблицата с *CarPage.List.Cars* и форма за редакция, и връзва бутоните към *Search.SearchAsync*, *Edit.StartNew*, *Edit.StartEdit*, *Edit.SaveAsync*, *Edit.DeleteAsync*.

Ключови типове

- ***RentACarDbContext*** – главният клас за достъп до базата; описва таблиците, релациите между *Car*, *Customer*, *Location* и *Rental*.
- ***DbContextFactory*** – капсулира избора на СУБД; чрез свойството *Provider* решава дали да конфигурира контекста за *SQL Server LocalDB* или за *SQLite*.
- ***DbPathHelper*** – решава конкретния практически проблем с общия *SQLite* файл, като връща път към *SharedData\rentacar.db* независимо дали извикването идва от *WPF* или от *Blazor*.
- ***UnitOfWork*** – събира в себе си всички репозитории и гарантира, че промяната по няколко таблици може да се запише с един *SaveChangesAsync()*.
- ***CarPageViewModel*** – координира *ViewModel*-ите за търсене, листване на списъците и редакция и предоставя единен интерфейс за „*Cars*“ страницата, както за *WPF*, така и за *Blazor*.
- ***CarSearchViewModel*** – държи състоянието на филтрите и знае как да извика репозиторията, така че да върне правилния списък от коли.
- ***CarListViewModel*** – отговаря за текущия списък от коли и избора на конкретна кола, която да бъде редактирана или изтрита.

- ***CarEditViewModel*** – държи логиката за създаване и редакция на кола, включително свързването с избрана *Location* и решението дали да се добавя нов запис или да се запази съществуващ.

Преизползване на **ViewModel**-ите

Преизползването на *ViewModel*-ите е реализирано така, че същите класове работят без промяна и в *WPF*, и в *Blazor*.

➤ В *WPF*:

- *MainWindow* създава *MainViewModel*, който вътре съдържа *CarPageViewModel*.
- *CarPageView* при *Loaded* сменя своя *DataContext* към *MainViewModel.CarPage*, след което *XAML*-ът и *code-behind* виждат *CarPageViewModel* и работят с него.

➤ В *Blazor*:

- В *Program.cs* *CarPageViewModel* се регистрира като *Scoped* услуга.
- В *Cars.razor* се използва *@inject CarPageViewModel CarPage*, което дава същия *ViewModel* като в *WPF*.
- Компонентът извиква *CarPage.LoadAsync()*, *CarPage.Search.SearchAsync()*, *CarPage.Edit.StartNew()*, *CarPage.Edit.SaveAsync()*, *CarPage.Edit.DeleteAsync()* по същия начин, по който *WPF* бутоните ги викат през *code-behind*.

Преизползването на СУБД

Преизползване на базата данни е решено на две нива:

- Общ *EF Core* модел и *DbContext* – и двете приложения реферират към един и същ проект *RentACar.Model*, който съдържа *RentACarDbContext*, *ентити-а* и *репозитории*.
- Общ *SQLite* файл – чрез *DbPathHelper.GetSharedSqlitePath()* и двете приложения получават път към една и съща *SharedData\rentacar.db*, независимо от собствените си *bin* директории.

За избор на СУБД се използва свойството *DbContextFactory.Provider*:

- Ако е *"SqlServer"*, *DbContextFactory.Create()* конфигурира контекста с *UseSqlServer(...)* и използва *LocalDB*.
- Ако е *"Sqlite"*, използва *UseSqlite(...)* и пътя от *DbPathHelper*.

В WPF този избор се сменя от *UI* (бутон/*combobox* в *MainWindow*), който променя *Provider*, създава нов контекст през фабриката и презарежда *ViewModel-ume*. В *Blazor* същата идея може да се приложи чрез избор на база от навигацията (*NavBar*).

Ключови откъси от кода

DbContextFactory - избор на СУБД в един централен метод. Този клас показва как с едно свойство *Provider* може да се превключва между *SQLite* и *SQL Server*, без да се пипа никъде другаде по кода.

```
public static class DbContextFactory
{
    public static string Provider { get; set; } = "Sqlite";

    public static RentACarDbContext Create()
    {
        var optionsBuilder = new DbContextOptionsBuilder<RentACarDbContext>();

        if (Provider == "SqlServer")
        {
            var cs =
                "Server=(localdb)\\MSSQLLocalDB;Database=RentACarDb;Trusted_Connection=True;MultipleActiveResultSets=True;";
            optionsBuilder.UseSqlServer(cs);
        }
        else // Sqlite
        {
            var dbPath = DbPathHelper.GetSharedSqlitePath();
            var cs = $"Data Source={dbPath}";
            optionsBuilder.UseSqlite(cs);
        }

        return new RentACarDbContext(optionsBuilder.Options);
    }
}
```

DbPathHelper – общ *SQLite* файл за *WPF* и *Blazor*.

```
public static class DbPathHelper
{
    public static string GetSharedSqlitePath()
    {
        var baseDir = AppDomain.CurrentDomain.BaseDirectory;
        var solutionRoot = Directory.GetParent(baseDir)!.Parent!.Parent!.FullName;

        var dataDir = Path.Combine(solutionRoot, "SharedData");
        Directory.CreateDirectory(dataDir);

        return Path.Combine(dataDir, "rentacar.db");
    }
}
```

CarEditViewModel – логика за нова/редакция/изтриване на кола.

```
public class CarEditViewModel : INotifyPropertyChanged
{
    private readonly IUnitOfWork _unitOfWork;

    public ObservableCollection<Location> Locations { get; } = new();

    public CarEditViewModel(IUnitOfWork unitOfWork)
    {
        _unitOfWork = unitOfWork;
    }

    public async Task LoadLocationsAsync()
    {
        Locations.Clear();
        var all = await _unitOfWork.Locations.GetAllAsync();
        foreach (var loc in all)
            Locations.Add(loc);
    }

    . . .

    public void StartNew()
    {
        Current = new Car { Status = "Available" };
        SelectedLocation = Locations.FirstOrDefault();
    }

    public void StartEdit(Car car)
    {
        . . .
    }

    public async Task SaveAsync()
    {
        if (Current == null) return;

        if (SelectedLocation != null)
            Current.LocationId = SelectedLocation.Id;

        if (Current.Id == 0)
            await _unitOfWork.Cars.AddAsync(Current);

        await _unitOfWork.SaveChangesAsync();
    }

    public async Task DeleteAsync()
    {
        . . .
    }
}
```

CarPageView.xaml.cs – WPF View, който говори само с *ViewModel*

```
public partial class CarPageView : UserControl
{
    public CarPageView()
    {
        InitializeComponent();

        this.Loaded += (s, e) =>
        {
            if (DataContext is MainViewModel mainVm)
            {
                DataContext = mainVm.CarPage;
            }
        };
    }

    private async void SearchButton_Click(object sender, RoutedEventArgs e)
    {
        if (DataContext is CarPageViewModel vm)
            await vm.Search.SearchAsync();
    }

    private void NewCarButton_Click(object sender, RoutedEventArgs e)
    {
        . . .
    }

    . . .

    private async void SaveCarButton_Click(object sender, RoutedEventArgs e)
    {
        if (DataContext is CarPageViewModel vm)
        {
            await vm.Edit.SaveAsync();
            await vm.LoadAsync();
        }
    }

    private async void DeleteCarButton_Click(object sender, RoutedEventArgs e)
    {
        . . .
    }
}
```

Blazor Program.cs – DI и общ *DbContext* за уеб UI. Използване на *Model* и *ViewModel* слоевете в *Blazor*:

```
var builder = WebApplication.CreateBuilder(args);

DbContextFactory.Provider = "Sqlite";
```

```

builder.Services.AddDbContext<RentACarDbContext>(options =>
{
    if (DbContextFactory.Provider == "SqlServer")
    {
        var cs = "Server=(localdb)\\MSSQLLocalDB;...
    }
    else
    {
        ...
    }
});

builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
builder.Services.AddScoped<CarPageViewModel>();

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<RentACarDbContext>();
    db.Database.EnsureCreated();
}

```

Пример за използване на функционалностите

Добавяне на нов запис

The screenshot displays the 'Car WPF' application interface. At the top, there are tabs for 'Cars', 'Rentals', and 'Customers', with 'Cars' selected. A 'Database' dropdown menu is set to 'SQLite'. Below the tabs, there are input fields for 'Category', 'Status' (set to 'Available'), 'Min Rate', and 'Max Rate', followed by a 'Search' button. The main area features a table with columns: Reg. No., Make, Model, Category, Daily rate, Status, and Location. The first row of data is: CA8642PB, Dacia, Sandero, Economy, 20.0, Available, Sofia - Airport (SOF). To the right of the table is the 'Car editor' section, which includes buttons for 'New', 'Edit selected', 'Save', and 'Delete'. Below these buttons are input fields for 'Reg. No.', 'Make', 'Model', 'Category', 'Daily rate' (set to 0), 'Status' (set to 'Available'), and 'Location' (set to 'Plovdiv'). At the bottom right, the 'Selected car details' section shows the details of the selected car: CA8642PB, Dacia, Sandero, Economy, 20.0, Available.

Фиг.1 - Екран Car WPF

Rental editor ¹

New rental Edit selected Save

Pickup: ² 4/26/2026 15

Dropoff: 4/30/2026 15

Total price: 80.0

Status: ³ Planned

Car: ⁴ CA8642PB

Customer: DL-21131

автоматично изчислено

4 дни * 20 = 80.0

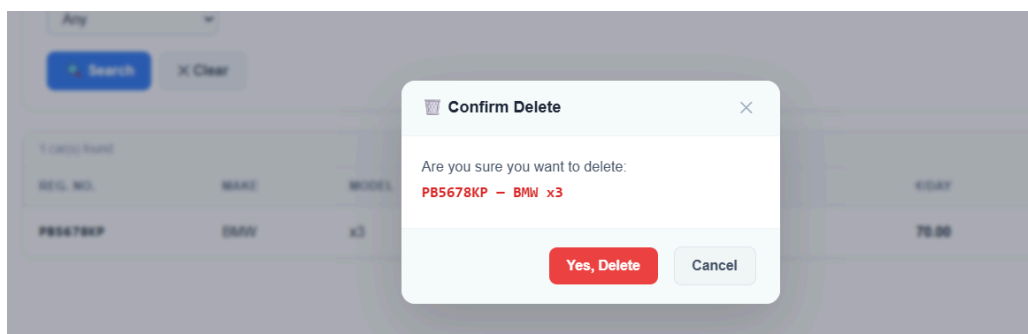
Selected car details

CA8642PB
Dacia
Sandero
Economy
20.0
Available

Full name	Driving license	Phone	Email
Vasilena Milchova	DL-21131	0879544532	v.milchova@gmail.com

Фиг.2 - Добавяне на Rental в WPF с обяснение

Премахване на запис



Фиг.3 - Премахване на запис в Blazor

Full name	Driving license	Phone	Email
Vasilena Milchova	DL-21131	0879544532	v.milchova@gmail.com

Customer editor

New Edit se

Full name:

Driving license:

Phone:

Email:

Confirm delete

Do you really want to delete client - "Vasilena Milchova"?

Yes No

Фиг.4 -
Изтриване на
запис в Blazor

Редактиране на запис

Customers + New Customer

NAME: e.g. Ivan Petrov EMAIL: e.g. ivan@mail.com PHONE: e.g. +359...

Search X Clear

Edit Customer X

FULL NAME: Vasilena Milchova EMAIL: v.milchova@gmail.com PHONE: 0879544532 DRIVING LICENSE NO.: DL-21131

Save Cancel

1 customer(s) found

FULL NAME	EMAIL	PHONE	DRIVING LICENSE	ACTIONS
Vasilena Milchova	v.milchova@gmail.com	0879544532	DL-21131	✎ 🗑

Фиг.5 - Редактиране на запис в Blazor

Филтриране на записите

Cars + New Car

MAKE: e.g. Toyota MODEL: e.g. Corolla CATEGORY: SUV STATUS: All MIN RATE (€/DAY): MAX RATE (€/DAY):

LOCATION: Any

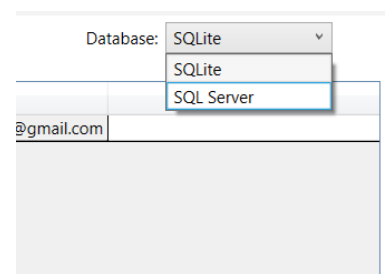
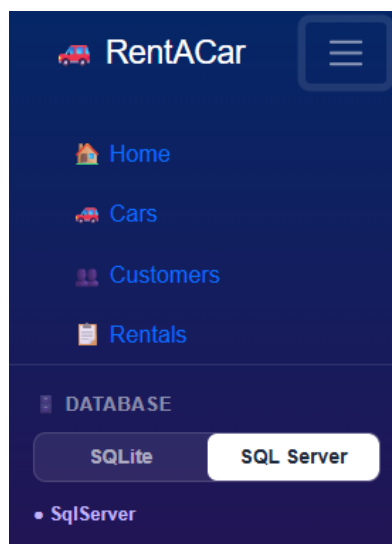
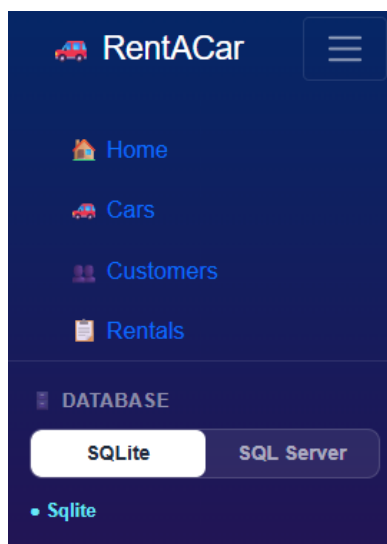
Search X Clear

1 car(s) found

REG. NO.	MAKE	MODEL	CATEGORY	STATUS	€/DAY	LOCATION	ACTIONS
PB5678KP	BMW	x3	SUV	Maintenance	70.00	Plovdiv	✎ 🗑

Фиг.6 - Сортиране, показано в Blazor

Смяна на базата данни



Фиг.7, 8 и 9 - Промяна на базата данни.

Фиг 7 и 8 - Blazor

Фиг 9 - WPF

(от ляво надясно)

Заклучение

С един общ *Model* слой беше възможно да се изгради както *WPF* десктоп приложение, така и *Blazor Server* уеб приложение, които работят върху един и същи бази данни. Работата по проекта помогна да се разбере по-добре няколко ключови технологии: *Entity Framework Core* с повече от един *provider*, *dependency injection* в *ASP.NET Core/Blazor* и *MVVM* подходът във *WPF*. От гледна точка на *UI* технологиите, *WPF* се оказа по-сложен заради *XAML* и нуждата от повече усилия за дизайн, докато *Blazor* е по-лесен и гъвкав начин да се изгради уеб интерфейс с *HTML-подобен* синтаксис и същите *ViewModel*-и. В бъдеще този проект може да бъде развит като се добавят по-добри проверки за грешки и проверки при попълване на данните. Както и разширение на поддържаните функционалности.

Източници

- [1] [Code-Behind and XAML - WPF | Microsoft Learn](#)
- [2] [Using ObservableCollection in C# \(Beginner-Friendly Guide\)](#)
- [3] [Overview of Entity Framework Core - EF Core | Microsoft Learn](#)
- [4] [Dependency injection - .NET | Microsoft Learn](#)
- [5] [RentACar GitHub](#)

При изготвянето на кода, беше използван [LLM асистент](#) в режим, ориентиран към обучение „стъпка по стъпка“ (*learn step-by-step mode*), а не като инструмент за автоматично генериране на цялото решение. В процеса на работа асистентът помагаше основно с: изясняване на концепции, предлагане на примерни фрагменти код, които след това се адаптираха, променяха и дебъгваха ръчно.